

AFRILANG Stdlib

std/c/nlpbow

Français. Bibliothèque AFRILANG 'std/c/nlpbow' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bowLen, bowUpper, bowLower, bowTrim, bowSplit, bowJoin, bowReplace.

```
'import "std/c/nlpbow"' · 'use nlpbow'
```

```
- 'bowLen(s text) → number' - 'bowUpper(s text) → text' - 'bowLower(s text) → text' - 'bowTrim(s text) → text' - 'bowSplit(s text, delim text) → list text' - 'bowJoin(parts list text, delim text) → text' - 'bowReplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/nlp-bow' (tier complex, category complex). Exports 7 function(s): bowLen, bowUpper, bowLower, bowTrim, bowSplit, bowJoin, bowReplace.

```
'import "std/c/nlpbow"' · 'use nlpbow'
```

```
- 'bowLen(s text) → number' - 'bowUpper(s text) → text' - 'bowLower(s text) → text' - 'bowTrim(s text) → text' - 'bowSplit(s text, delim text) → list text' - 'bowJoin(parts list text, delim text) → text' - 'bowReplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlpbow' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bowLen, bowUpper, bowLower, bowTrim, bowSplit, bowJoin, bowReplace.

```
'import "std/c/nlpbow"' · 'use nlpbow'
```

```
- `bowLen(s text) → number`
- `bowUpper(s text) → text`
- `bowLower(s text) → text`
- `bowTrim(s text) → text`
- `bowSplit(s text, delim text) → list text`
- `bowJoin(parts list text, delim text) → text`
- `bowReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlpbow"
use nlpbow
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- bowLen(s text) → number•
- bowUpper(s text) → text•
- bowLower(s text) → text•
- bowTrim(s text) → text•
- bowSplit(s text, delim text) → list•
- bowJoin(parts list text, delim text) → text•
- bowReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/nlpbow"
use nlpbow
```

```
create result = bowLen(/* args */)
say result
```

```
create result = bowUpper(/* args */)
say result
```

```
create result = bowLower(/* args */)
say result
```

```
create result = bowTrim(/* args */)
say result
```

```
create result = bowSplit(/* args */)
say result
```

```
create result = bowJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlpbow"`` en tête de fichier.
- Lancez ``afri-lang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module `nlpbow`

```
function bowLen(s text) returns number
end
```

```
function bowUpper(s text) returns text
end
```

```
function bowLower(s text) returns text
end
```

```
function bowTrim(s text) returns text
end
```

```
function bowSplit(s text, delim text) returns list text
end
```

```
function bowJoin(parts list text, delim text) returns text
end
```

```
function bowReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/nlpbow' (tier complex, category complex). Exports 7 function(s): bowLen, bowUpper, bowLower, bowTrim, bowSplit, bowJoin, bowReplace.

'import "std/c/nlpbow"' · 'use nlpbow'

```
- `bowLen(s text) → number`  
- `bowUpper(s text) → text`  
- `bowLower(s text) → text`  
- `bowTrim(s text) → text`  
- `bowSplit(s text, delim text) → list text`  
- `bowJoin(parts list text, delim text) → text`  
- `bowReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlpbow"  
use nlpbow
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- bowLen(s text) → number•
bowUpper(s text) → text•
bowLower(s text) → text•
bowTrim(s text) → text•
bowSplit(s text, delim text) → list•
bowJoin(parts list text, delim text) → text•
bowReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/nlpbow"  
use nlpbow
```

```
create result = bowLen(/* args */)   
say result
```

```
create result = bowUpper(/* args */)   
say result
```

```
create result = bowLower(/* args */)   
say result
```

```
create result = bowTrim(/* args */)   
say result
```

```
create result = bowSplit(/* args */)   
say result
```

```
create result = bowJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlpbow"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlpbow

```
function bowLen(s text) returns number
end
```

```
function bowUpper(s text) returns text
end
```

```
function bowLower(s text) returns text
end
```

```
function bowTrim(s text) returns text
end
```

```
function bowSplit(s text, delim text) returns list text
end
```

```
function bowJoin(parts list text, delim text) returns text
end
```

```
function bowReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlpbow"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlpbow/>

Source (extrait)

```
module nlpbow

function bowLen(s text) returns number
end

function bowUpper(s text) returns text
```

```
end

function bowLower(s text) returns text
end

function bowTrim(s text) returns text
end

function bowSplit(s text, delim text) returns list text
end

function bowJoin(parts list text, delim text) returns text
end

function bowReplace(s text, from text, to text) returns text
end
end
```